

第7章 文件与 IO

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:熟悉 C 的 fopen/fread/fwrite/fclose、FILE*、errno 与标准流 stdin/stdout

🎯 本章学习目标

- 理解 C 的 FILE* 世界与 Rust 的 **Result + RAII** 世界的对应关系:打开失败不再返回 NULL,而是 Result
- 掌握路径类型 Path/PathBuf:为什么不能用 String 拼接路径,join 与 display 的正确用法
- 掌握文件读写的三种姿势:一次性读入、流式按行读、二进制读写,以及各自的适用场景
- 掌握 BufReader/BufWriter 缓冲语义,与 C 的 setvbuf/fread 缓冲机制对照
- 掌握 OpenOptions 的精细控制:创建/追加/截断,对应 C 的 "w"/"a"/"a+" 模式
- 掌握目录遍历 read_dir 与递归删除/创建,对照 C 的 opendir/readdir 与平台 API
- 会用 serde_json 做结构化的序列化与反序列化,告别手写字符串解析
- 掌握标准流 stdin/stdout/stderr 与命令行参数,main 返回 Result 的错误传播约定

💡 从 C 到 Rust:本章主题如何迁移

C 的 IO 函数是"全局状态 + 错误码"的样板:fopen 失败返回 NULL(还要 errno 分辨原因),fread 返回"读取的字节数"(要自查是否等于请求数),fclose 失败还可能静默丢数据,忘了 fclose 就是泄漏。每个 FILE* 是"不透明指针",打开模式靠字符串 "rb"/"a+" 约定。这套 API 你用了十年,深知其中暗坑:缓冲区没刷、错误码没查、路径太长、跨平台换行.....

Rust 把同一件事重新组织成三个支柱:**第一,错误成为类型**——每个 IO 调用返回 Result,失败信息(路径、原因)随值携带,不再靠 errno 猜;? 运算符把"失败就提前返回"写成一行(第9章详述,本章先用)。**第二,资源由类型管理**——File 离开作用域自动关闭(RAII,第5章已铺垫),没有 fclose,没有泄漏,没有"双关"。**第三,路径是类型,不是字符串**——Path 区分于 String,把"拼接分隔符""检查存在""平台差异"收进类型系统。

对照着学最省力:本章每一节都从"C 是怎么写的"出发,给出 Rust 对应写法与差异表。你先看到 fopen 与 File::open 的对应,再看到 BufReader 与 setvbuf 的对应,然后看到 serde_json 如何终结手写解析——这就是本章的迁移路线。

📦 前置知识自查

- C 的文件 API:fopen 模式字符串、fread/fwrite 返回值、fseek/feof、errno 与 perror
- C 的标准流 stdin/stdout/stderr、printf/fprintf/sprintf 家族
- 第4章的 main 函数与 Result 返回约定、第5章的 RAII/Drop 概念
- 第2章的运算符与表达式(本章大量使用 ? 与解构)
- (可选)JSON 的基本结构:对象、数组、字符串、数字

本章学习路线

1. **第一阶段(约1小时):**§7.1–§7.2,文件打开与路径类型——先会用 `File::open/fs::write` 读写文件
2. **第二阶段(约1.5小时):**§7.3–§7.4,缓冲读写、`OpenOptions` 与目录操作——生产 IO 的主力
3. **第三阶段(约1.5小时):**§7.5–§7.7,`serde_json`、标准流与错误传播——把 IO 与数据类型打通
4. **验收标准:**不查文档能写一个"按行读日志、统计、写汇总"的工具,且每个 IO 调用都带 `?` 或显式处理

本章知识导图

- §7.1 `FILE*` 世界 vs `Result` 世界:打开与关闭
- §7.2 路径类型:`Path` 与 `PathBuf`
- §7.3 读写的三种姿势:一次性、流式、二进制
- §7.4 精细控制:`OpenOptions` 与缓冲语义
- §7.5 目录操作:遍历、创建、删除
- §7.6 结构化数据:`serde_json` 序列化
- §7.7 标准流、命令行参数与错误传播
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§7.1 FILE* 世界 vs Result 世界:打开与关闭

7.1.1 C 的样板代码

```
// C:每个文件操作都要查错误码,忘了就静默出错
#include <stdio.h>

int copy_file(const char *src, const char *dst) {
    FILE *in = fopen(src, "rb");
    if (in == NULL) {                // 打开失败:返回 NULL,errno 告诉原因
        perror("open src");
        return -1;
    }
    FILE *out = fopen(dst, "wb");
    if (out == NULL) {
        perror("open dst");
        fclose(in);                  // 别忘了关掉第一个!
        return -1;
    }

    char buf[4096];
    size_t n;
    while ((n = fread(buf, 1, sizeof buf, in)) > 0) {
        if (fwrite(buf, 1, n, out) != n) {
            perror("write");
            fclose(in); fclose(out);
            return -1;
        }
    }
    fclose(in); fclose(out);        // 每一处提前返回都要记得关
    return 0;
}
```

这段代码的每一行都在跟"错误"搏斗:NULL 检查、errno、fclose 的成对出现。少一个 fclose 是泄漏,少一个检查是静默错误。

7.1.2 Rust 的对应:Result 携带错误,Drop 自动关闭

```
// Rust:同样的文件复制,错误是类型,资源是 RAII
use std::fs;
use std::io;

fn copy_file(src: &str, dst: &str) -> io::Result<()> {
    let data = fs::read(src)?;          // 失败返回 Err,包含路径与原因
    fs::write(dst, data)?;              // 失败同样提前返回
    Ok(())
}

fn main() {
    match copy_file("a.txt", "b.txt") {
        Ok(()) => println!("复制成功"),
        Err(e) => eprintln!("复制失败:{e}"),    // e 里带着"哪个文件、什么错误"
    }
}
```

对照一目了然:

C	Rust	差异要点
fopen 返回 FILE* 或 NULL	File::open 返回 Result<File>	NULL 意味着"错误与类型混在一起";Result 让错误成为类型的一部分
errno + perror	io::Error(含路径、错误码、系统消息)	错误随值传递,不会因线程/调用被覆盖
fclose 手动调用	Drop 自动关闭	没有泄漏,没有 double-close
模式字符串 "rb"/"a+"	OpenOptions 方法链	模式编译期检查,拼错字符串?C 会静默失败,Rust 报编译错误
fread 返回值要自查	read_to_end/read_to_string 一次到位	读完没读全是内部逻辑,不留给调用方
feof/ferror 判断结束	read 返回 Ok(0) 或 Err	EOF 与错误用类型区分,不是状态位

⚠ 易错点 1:File::open 只能读,不能写

File::open(path) 等价 C 的 "rb"(只读)。想写文件、想追加、想创建,必须用 File::create(覆盖写,等价 "wb")或 OpenOptions(§7.4)。生产代码最常见的报错就是"用 File::open 打开后调用 write,得到 'cannot borrow as mutable' 或 PermissionDenied"——先想清楚你要哪种模式。

§7.2 路径类型:Path 与 PathBuf

7.2.1 为什么不用 String 表示路径

C 里路径就是 char 数组,拼接靠 sprintf("s%s/config.ini", dir)。Windows 与 Unix 分隔符不同、编码不同,同一段代码换个平台就崩。Rust 用两个类型解决:**Path**(只读借用,对应 &str)与 **PathBuf**(可增长的拥有型,对应 String)。分隔符由 join 自动处理,路径操作(取文件名、取父目录、加后缀)都是类型的方法:

```

use std::path::{Path, PathBuf};

fn main() {
    // C:char dir[256]; sprintf(dir, "%s/logs/app.log", base);
    let base = Path::new("/var/data");
    let log = base.join("logs").join("app.log");    // 自动拼分隔符
    println!("完整路径:{}", log.display());        // /var/data/logs/app.log

    // 取文件名与父目录(对照 basename/dirname)
    println!("文件名:{}", log.file_name().unwrap().to_string_lossy());
    println!("父目录:{}", log.parent().unwrap().display());

    // 换扩展名:生成另一个路径,原路径不动
    let bak = log.with_extension("log.bak");
    println!("备份路径:{}", bak.display());

    // 判断路径属性(对照 stat)
    let p = Path::new("Cargo.toml");
    println!("存在:{} 是文件:{} 是目录:{}",
        p.exists(), p.is_file(), p.is_dir());

    // 转为拥有型 PathBuf(用于长期保存)
    let owned: PathBuf = log.to_path_buf();
    println!("owned:{}", owned.display());
}

```

7.2.2 display 与 to_string_lossy

Path 可能不是合法 UTF-8(Unix 文件名可以是任意字节)。打印用 `display()` (按当前平台编码显示),转字符串用 `to_string_lossy()` (非法字节替换为 U+FFFD)。不要直接用 `to_str()`——它返回 `Option`,遇到非 UTF-8 是 `None`,生产代码里要先处理。

C 的路径操作	Rust 对应
<code>sprintf(dir, "%s/%s", base, name)</code>	<code>Path::join:base.join(name)</code>
<code>basename(3)</code>	<code>path.file_name() -> Option<&OsStr></code>
<code>dirname(3)</code>	<code>path.parent() -> Option<&Path></code>
<code>strchr(name, '.') 改扩展名</code>	<code>path.with_extension("txt")</code>
<code>stat()</code> 判断类型	<code>path.is_file() / is_dir() / exists()</code>
字符串拼出来的分隔符	<code>join</code> 自动按平台处理

§7.3 读写的三种姿势:一次性、流式、二进制

7.3.1 一次性读入:小文件首选

```
use std::fs;

fn main() -> std::io::Result<> {
    // 整个文件读成字符串(按 UTF-8 解码)
    let text = fs::read_to_string("config.toml"?);
    println!("配置内容:\n{}", text);

    // 整个文件读成字节(二进制或不确定编码)
    let bytes = fs::read("logo.png"?);
    println!("图片大小: {} 字节", bytes.len());

    // 一次写入(覆盖)
    fs::write("out.txt", "第一行\n第二行\n"?);

    Ok(())
}
```

适用场景:配置文件、日志单条、体积已知的小文件(经验阈值:几十 KB 以内)。大文件一次性读入会吃满内存——那就用流式。

7.3.2 流式按行读:BufReader + lines

```
use std::fs::File;
use std::io::{BufRead, BufReader};

fn main() -> std::io::Result<> {
    // C 的 getline 循环对应:BufReader 提供缓冲的按行迭代
    let file = File::open("access.log"?);
    let reader = BufReader::new(file);

    let mut line_no = 0u32;
    for line in reader.lines() {           // 逐行,不整文件进内存
        let line = line?;                 // 每一行都可能 IO 失败(解码/读盘)
        line_no += 1;
        if line.contains("ERROR") {
            println!("{line_no}: {line}");
        }
    }
    println!("共扫描 {line_no} 行");
    Ok(())
}
```

7.3.3 二进制流式读:read_exact 与循环

```
use std::fs::File;
use std::io::{Read};

fn main() -> std::io::Result<()> {
    let mut file = File::open("data.bin")?;

    // 固定大小结构:一次读满(对照 fread 后自查返回值)
    let mut header = [0u8; 16];
    file.read_exact(&mut header)?; // 读不够 16 字节直接报 UnexpectedEof
    println!("头 16 字节:{header:02X?}");

    // 长度不确定的数据:循环读,read 返回 0 表示 EOF
    let mut rest = Vec::new();
    file.read_to_end(&mut rest)?;
    println!("剩余 {} 字节", rest.len());
    Ok(())
}
```

⚠ 易错点 2:read 与 read_exact 的区别

`read(&mut buf)` 返回"实际读到的字节数",可能小于 `buf` 长度(非阻塞、缓冲边界、信号中断),必须循环;而 `read_exact(&mut buf)` 保证读满或返回错误——解析固定大小结构(文件头、报头、帧)永远用 `read_exact`,不要自己写循环。这是 C 的 `fread` 返回值检查在 Rust 里的类型化版本。

§7.4 精细控制:OpenOptions 与缓冲语义

7.4.1 OpenOptions:打开模式的类型化版本

```
use std::fs::OpenOptions;

fn main() -> std::io::Result<()> {
    // C: fopen(path, "a") — 追加写
    let mut log = OpenOptions::new()
        .append(true)           // 追加
        .create(true)           // 不存在则创建
        .open("app.log")?;
    use std::io::Write;
    writeln!(log, "[{}] 服务启动", "2026-08-26")?;

    // C: fopen(path, "w") — 截断覆盖
    let mut out = OpenOptions::new()
        .write(true)
        .create(true)
        .truncate(true)
        .open("report.txt")?;
    writeln!(out, "月度报告")?;

    // C: fopen(path, "r+") — 读写,不截断
    let _both = OpenOptions::new()
        .read(true)
        .write(true)
        .open("data.bin")?;

    Ok(())
}
```

7.4.2 BufWriter:攒批写,减少系统调用

```
use std::fs::File;
use std::io::{BufWriter, Write};

fn main() -> std::io::Result<()> {
    // 裸 File::write 每次调用都是一次系统调用;BufWriter 攒满再刷
    let file = File::create("big.txt")?;
    let mut w = BufWriter::new(file);

    for i in 0..100_000 {
        writeln!(w, "第 {i} 行数据")?;    // 先进内存缓冲
    }
    w.flush()?;                          // 最后统一落盘(或 Drop 时自动刷)
    println!("写完");
    Ok(())
}
```

与 C 对照: `setvbuf(stdout, NULL, _IOFBF, 4096)` 之后 `printf` 的行为,就是 `BufWriter` 的默认行为。区别: Rust 的缓冲是 **显式类型**——你拿到 `File` 就是无缓冲直写,想要缓冲就包一层 `BufWriter`,编译器帮你记账,不会出现"忘了 `setvbuf`

导致性能灾难"的隐式状态。

工程建议:日志/上报类写操作,顺序保证

崩溃恢复类场景(交易日志、追加式审计)要"写一条刷一条",不能攒批。做法:不用 `BufWriter` 直接写 `File`,或每行后显式 `flush()`;性能敏感且允许丢尾巴的场景(统计输出)才用 `BufWriter`。C 里同样纠结 `fflush`,但 Rust 把选择放在了明面上。

§7.5 目录操作:遍历、创建、删除

7.5.1 列目录与递归遍历

```
use std::fs;

fn main() -> std::io::Result<()> {
    // 对照 C:opendir/readdir/closedir
    println!("== 当前目录内容 ==");
    for entry in fs::read_dir(".")? {
        let entry = entry?;           // 每个条目本身可能出错
        let name = entry.file_name(); // OsString:不是 UTF-8 也安全
        println!("{}", name.to_string_lossy());
    }

    // 递归统计:目录树中所有 .rs 文件的行数(手写递归,对照 C 的递归遍历)
    fn count_rs_lines(dir: &std::path::Path) -> std::io::Result<(u32, u32)> {
        let mut files = 0u32;
        let mut lines = 0u32;
        for entry in fs::read_dir(dir)? {
            let entry = entry?;
            let path = entry.path();
            if path.is_dir() {
                let (f, l) = count_rs_lines(&path)?; // 递归
                files += f;
                lines += l;
            } else if path.extension().and_then(|e| e.to_str()) == Some("rs") {
                files += 1;
                lines += std::io::BufRead::lines(
                    std::io::BufReader::new(fs::File::open(&path)?)
                ).count() as u32;
            }
        }
        Ok((files, lines))
    }

    let (f, l) = count_rs_lines(std::path::Path::new("src"))?;
    println!("src 下 {f} 个 rs 文件,共 {l} 行");
    Ok(())
}
```

7.5.2 创建与删除

```
use std::fs;

fn main() -> std::io::Result<()> {
    // 递归创建目录(对照 mkdir -p)
    fs::create_dir_all("backup/2026/08")?;

    // 删除文件
    fs::remove_file("tmp/old.tmp")?;

    // 递归删除目录树(危险操作,生产慎用)
    fs::remove_dir_all("tmp")?;

    // 重命名/移动
    fs::rename("a.txt", "archive/a.txt")?;

    println!("目录操作完成");
    Ok(())
}
```

C / POSIX	Rust std::fs
opendir / readdir / closedir	fs::read_dir 迭代器(自动关闭)
mkdir -p(循环建)	fs::create_dir_all
rmdir	fs::remove_dir
unlink	fs::remove_file
rm -rf(手写递归)	fs::remove_dir_all
rename(2)	fs::rename
stat 判断类型	path.is_file() / is_dir() / metadata()

⚠ 易错点 3:is_dir() 与 read_dir 的组合陷阱

`path.is_dir()` 本身做一次系统调用;在遍历中, `entry.path()` 之后再调 `is_dir` 是"先取路径再 stat"。更高效且避免 TOCTOU(时间差攻击)的做法:用 `entry.file_type()` 的 `is_dir()`,它来自 `readdir` 已经拿到的事实。另外注意:文件系统是活的——遍历中途文件被删,`is_dir` 可能返回 `false`,别把它当权威,IO 错误要留好? 出口。

§7.6 结构化数据:serde_json 序列化

7.6.1 C 的世界:手写解析,苦不堪言

C 读 JSON 要么用第三方库,要么手写字符串切割。手写的典型:strstr 找冒号、atoi 转数字、转义字符自己处理——每加一个字段改一处解析。Rust 的标准做法:serde 框架 + serde_json, 结构体上写 derive,序列化与反序列化自动生成 (derive 原理第10/12章详解,这里直接用):

```

use serde::{Deserialize, Serialize};

// 一个 derive,换来 to_json 与 from_json 两套能力
#[derive(Debug, Serialize, Deserialize)]
struct Config {
    host: String,
    port: u16,
    workers: u32,
    tags: Vec<String>,          // 数组字段
    debug: bool,               // 布尔字段
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let cfg = Config {
        host: "0.0.0.0".into(),
        port: 8080,
        workers: 4,
        tags: vec!["api".into(), "prod".into()],
        debug: false,
    };

    // 序列化:结构体 -> JSON 字符串(pretty 便于人类阅读)
    let json = serde_json::to_string_pretty(&cfg)?;
    println!("{}", json);

    // 反序列化:JSON 字符串 -> 结构体;字段缺失/类型不对直接报错
    let text = r#"{"host":"127.0.0.1","port":9090,"workers":2,
        "tags":["dev"],"debug":true}"#;
    let cfg2: Config = serde_json::from_str(text)?;
    println!("解析结果:{cfg2:?}");

    // 宽松场景:先解析成通用值再取字段
    let v: serde_json::Value = serde_json::from_str(text)?;
    println!("host = {}", v["host"]);
    Ok(())
}

```

工程建议:serde 是事实标准,不是"框架绑架"

serde + serde_json 在 crates.io 上是 Rust 生态的基石级依赖(几乎所有网络、配置、数据库库都在用)。Cargo.toml 加两行:serde = { version = "1", features = ["derive"] } 与 serde_json = "1"。字段命名习惯是 snake_case,对接外部 camelCase 的 JSON 时用 #[serde(rename_all = "camelCase")] 或逐字段 #[serde(rename = "...")]。

§7.7 标准流、命令行参数与错误传播

7.7.1 stdin/stdout/stderr 与命令行参数

```
use std::env;
use std::io::{self, BufRead, Write};

fn main() -> io::Result<()> {
    // 命令行参数:对照 C 的 argv[1..];env::args() 第一个是程序名
    let args: Vec<String> = env::args().collect();
    if args.len() < 2 {
        eprintln!("用法:{<关键词>", args[0]); // 标准错误,对照 fprintf(stderr,...)
        std::process::exit(1);                // 对照 exit(1)
    }
    let keyword = &args[1];

    // 按行读 stdin:对照 getline(stdin) 循环
    let stdin = io::stdin();
    for line in stdin.lock().lines() {
        let line = line?;
        if line.contains(keyword) {
            // 显式写 stdout(对照 fputs(stdout)),flush 保证立即输出
            let mut out = io::stdout().lock();
            writeln!(out, "{line}")?;
        }
    }
    Ok(())
}
```

7.7.2 main 返回 Result:错误传播的完整链路

生产约定的主力形态:main 返回 `Result<(), Box<dyn std::error::Error>>`,所有 IO 调用一路 `?` 到底。程序失败时,系统打印错误信息并以非零码退出——这替代了 C 里"每个函数返回 int + 逐层 if 检查 + 顶层 perror"的全套仪式:

```

use std::fs;
use std::io::{self, BufRead, BufReader};

// 读文件并统计非空行数;失败信息随类型一路传播
fn count_nonempty(path: &str) -> io::Result<u32> {
    let reader = BufReader::new(fs::File::open(path)?);
    let mut n = 0;
    for line in reader.lines() {
        let line = line?;
        if !line.trim().is_empty() {
            n += 1;
        }
    }
    Ok(n)
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let path = "notes.txt";
    let n = count_nonempty(path)?;      // 打开失败:整条链快速返回
    println!("{path} 非空行数:{n}");
    Ok(())
}

// 运行:./tool; 若 notes.txt 不存在,输出:
// Error: Os { code: 2, kind: NotFound, message: "系统找不到指定的文件。" }
// 退出码非 0 — 与 C 的 exit(EXIT_FAILURE) 语义一致,但信息完整

```

C	Rust
printf / puts	println!
fprintf(stderr, ...)	eprintln!
fprintf(stdout, ...) + fflush	write!(io::stdout(), ...)? 或 println!
getchar / getline(stdin)	io::stdin().lock().lines() / read_line
argv / argc	env::args()(跳过程序名后即 argv[1..])
exit(1) + perror	std::process::exit(1) 或 main 返回 Err(自动打印)
errno 跨函数传递	? 把 io::Error 一路向上,类型明确

本章速查表

主题	结论 / 写法
打开只读	<code>File::open(path)? -> Result<File></code> (等价 <code>"rb"</code>)
覆盖写	<code>fs::write(path, data)?</code> 或 <code>File::create(path)?</code> (等价 <code>"wb"</code>)
追加写	<code>OpenOptions::new().append(true).create(true).open(path)?</code>
一次性读文本	<code>fs::read_to_string(path)?</code> (UTF-8,小文件)
一次性读字节	<code>fs::read(path)? -> Vec<u8></code>
按行流式读	<code>BufReader::new(File::open(p)?.lines())</code>
二进制定长读	<code>file.read_exact(&mut [u8; N])?</code> (读不满报 <code>UnexpectedEof</code>)
缓冲写	<code>BufWriter::new(File::create(p)?);</code> 写完 <code>flush()</code> 或 <code>Drop</code> 自动刷
路径拼接	<code>Path::new(base).join("a").join("b");</code> 打印用 <code>display()</code>
取文件名/父目录	<code>path.file_name()</code> / <code>path.parent()</code> (返回 <code>Option</code>)
换扩展名	<code>path.with_extension("bak")</code>
列目录	<code>for entry in fs::read_dir(dir)? { let e = entry?; }</code>
递归建目录	<code>fs::create_dir_all("a/b/c")?(<code>mkdir -p</code>)</code>
删除/移动	<code>fs::remove_file</code> / <code>fs::remove_dir_all</code> / <code>fs::rename</code>
JSON 序列化	<code>#[derive(Serialize, Deserialize)] + serde_json::to_string / from_str</code>
命令行参数	<code>env::args().collect::<Vec<_>>()</code> ; <code>args[0]</code> 是程序名
标准错误	<code>eprintln!("{}", ...);</code> 退出码: <code>std::process::exit(1)</code>
main 错误出口	<code>fn main() -> Result<>, Box<dyn std::error::Error>> + 全程 ?</code>

最小必会清单

- 能说出 `File::open` 与 `File::create`、`OpenOptions` 三种打开方式分别对应 C 的哪些模式
- 能写出"读文件 -> 处理 -> 写文件"的完整主链,且每个 IO 调用要么 ? 要么显式 `match`
- 知道小文件用 `read_to_string/read`,大文件用 `BufReader` 流式;能解释内存差异
- 知道 `read_exact` 与 `read` 的区别,固定结构解析必用 `read_exact`
- 能徒手写出 `BufWriter` 的"创建 -> 循环写 -> `flush`"三段式
- 能用 `Path::join` 拼接路径,知道为什么不用字符串拼,知道 `display()` 与 `to_string_lossy`
- 能写 `read_dir` 遍历并递归统计目录树;知道 `file_type()` 比 `is_dir()` 少一次 `stat`
- 能用 `serde derive` 定义配置结构体, `from_str` 与 `to_string_pretty` 双向转换
- 能写出 `main -> Result` 的错误传播主链,与 C 的逐层 `errno` 检查对照
- 能说出 `RAII` 自动关闭如何消灭 `fclose` 遗漏与 `double-close` 两类 bug

自测题

Q 1. `File::open("x.txt")` 失败时,你能拿到什么信息?与 C 的 `fopen` 返回 `NULL + errno` 有什么本质不同?

✓ 参考答案

拿到一个 `io::Error` 值,包含:错误种类(`NotFound/PermissionDenied` 等)、系统错误码、可读消息(带路径)。本质不同:错误是值的一部分,随调用链传递,不会被其他调用覆盖;`NULL` 只是一个哨兵,细节全靠 `errno` 这个全局状态。Rust 的 `Err` 可以一路?传播,在顶层统一打印,中途不需要逐层检查。

Q 2. 一次性读入与流式按行读,各自适用什么场景?判断依据是什么?

✓ 参考答案

一次性(`read_to_string/read`)适合小文件(经验阈值几十 KB 内)、需要整份内容做处理(解析、替换)的场景,代码最短;流式(`BufReader.lines()`)适合大文件(日志、数据导出),内存占用与文件大小无关,只与单行长度有关。判断依据:文件是否可能超过内存预算、是否需要全部内容同时在场。

Q 3. `read(buf)` 与 `read_exact(buf)` 有什么区别?什么场景必须用 `read_exact`?

✓ 参考答案

`read` 返回实际读到的字节数(可能小于请求数,如缓冲边界、非阻塞),需要循环直至读满;`read_exact` 保证填满缓冲区或返回错误(`UnexpectedEof`)。解析定长结构(文件头、帧头、固定记录)必须用 `read_exact`——C 的“`fread` 后自查返回值”被吸收进了类型保证,杜绝“结构体只读了一半”的静默 bug。

Q 4. 用字符串直接拼路径会出什么问题?`Path::join` 解决了哪些?

✓ 参考答案

问题:分隔符需自己处理(Windows 用 `\` 而 Unix 用 `/`),拼接时容易出“多一个少一个斜杠”,非 UTF-8 文件名无法用 `String` 表示。`join` 解决:按平台自动插入正确分隔符;`Path` 内部可容纳非 UTF-8(`OsStr`);`display()` 提供安全的显示转换。路径从此是类型,不是文本。

Q 5. `serde` 的 `derive` 相比手写字符串解析,在生产上赢了什么?

✓ 参考答案

三点:① 正确性——转义、嵌套、数字边界、类型不匹配全部由库处理并给出错误,手写解析的漏洞(越界、转义漏处理)不复存在;② 维护成本——结构体加字段,序列化自动跟进,手写解析要同步改两处;③ 类型安全——`from_str` 直接产出强类型结构体,后续代码不用再手工转换与检查。

Q 6. 一个程序既要“正常输出到终端”,又要“把日志写进文件”,用 Rust 怎么写?

✓ 参考答案

双流并存:println!/write!(stdout) 走标准输出;日志用 OpenOptions 追加模式打开文件 + BufWriter(需要立即落盘则每次 flush),eprintln! 走标准错误。与 C 的 stdout/stderr 分工一致,只是文件句柄是 Result 而来、作用域结束自动关闭。把"输出目的地"抽成 trait 或闭包,就能实现"同一条消息双写"的典型架构(第6章 trait 的用武之地)。

章末实验题:日志解析统计工具

实验 7:日志解析统计工具

实验目标:写一个命令行工具,读入一个日志文件,统计各级别(INFO/WARN/ERROR)的条数,把 ERROR 详情与统计汇总写入输出文件。

实验要求:

- 任务 1:命令行参数解析——程序名、输入路径、输出路径(覆盖:env::args §7.7)
- 任务 2:输入路径存在性检查,不存在则打印错误并退出非零(覆盖:Path 操作 §7.2)
- 任务 3:流式按行读入,解析"时间戳 级别 消息"格式(覆盖:BufReader §7.3)
- 任务 4:HashMap 统计各级别条数;收集 ERROR 行(覆盖:数据组织)
- 任务 5:汇总与错误详情写入输出文件(覆盖:BufWriter/fs::write §7.3)
- 任务 6:全程错误传播——main 返回 Result,打开失败信息可见(覆盖:错误传播 §7.7)
- 加分:用 serde_json 把统计结果序列化后写入(覆盖:serde_json §7.6)

日志样例行: 2026-08-26 10:00:01 INFO 服务启动、 2026-08-26 10:01:22 ERROR 数据库连接超时

第一步 一步:cargo new logstat;先在 main 里用 env::args 拿到输入输出路径

第二步 二步:写 parse_line 函数,把一行拆成(时间戳, 级别, 消息)

第三步 三步:搭起读入循环,BufReader.lines() 逐行调用 parse_line,先用 println 验证解析

第四步 四步:加 HashMap 统计与 ERROR 收集

第五步 五步:写汇总文件,跑通全流程;再把 parse_line 的返回值改成 Result,处理坏行

参考答案要点


```
// ===== 实验 7:日志解析统计工具 =====
// 覆盖知识点:命令行参数($7.7)、路径检查($7.2)、流式读($7.3)、
// 缓冲写($7.3)、错误传播($7.7)、serde_json 汇总($7.6)

use std::collections::HashMap;
use std::env;
use std::fs;
use std::io::{BufRead, BufReader, BufWriter, Write};
use std::path::Path;

/// 日志行解析结果
#[derive(Debug)]
struct LogLine {
    timestamp: String,
    level: String,
    message: String,
}

/// 解析一行:"2026-08-26 10:00:01 INFO 服务启动"
/// 时间戳含空格,所以从右侧切:最后两个字段是"级别 消息"
fn parse_line(line: &str) -> Option<LogLine> {
    let line = line.trim();
    if line.is_empty() {
        return None;
    }
    // 前 19 个字符是 "YYYY-MM-DD HH:MM:SS"
    if line.len() < 20 {
        return None;
    }
    let timestamp = line[..19].to_string();
    let rest = &line[20..]; // "INFO 服务启动"
    let mut parts = rest.splitn(2, ' '); // 只拆一次:级别 + 整段消息
    let level = parts.next()?.to_string();
    let message = parts.next()?.to_string();
    Some(LogLine { timestamp, level, message })
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // ---- 任务 1:命令行参数 ----
    let args: Vec<String> = env::args().collect();
    if args.len() != 3 {
        eprintln!("用法:{<输入日志> <输出汇总>", args[0]);
        std::process::exit(2);
    }
    let in_path = &args[1];
    let out_path = &args[2];

    // ---- 任务 2:输入存在性检查 ----
    if !Path::new(in_path).is_file() {
        eprintln!("错误:找不到日志文件 {in_path}");
        std::process::exit(1);
    }

    // ---- 任务 3 + 4:流式读入与统计 ----
    let reader = BufReader::new(fs::File::open(in_path)?);
```

```

let mut counts: HashMap<String, u32> = HashMap::new();
let mut errors: Vec<LogLine> = Vec::new();
let mut bad_lines = 0u32;
let mut total = 0u32;

for line in reader.lines() {
    let line = line?; // 读行失败:传播
    match parse_line(&line) {
        Some(log) => {
            total += 1;
            *counts.entry(log.level.clone()).or_insert(0) += 1;
            if log.level == "ERROR" {
                errors.push(log);
            }
        }
        None => bad_lines += 1, // 坏行:计数,不中断
    }
}

// ---- 任务 5:写汇总文件 ----
let file = fs::File::create(out_path)?;
let mut w = BufWriter::new(file);

writeln!(w, "=== 日志统计汇总 ===");
writeln!(w, "输入文件:{}, in_path)?;
writeln!(w, "总行数:{total} 坏行:{bad_lines}");
writeln!(w, "--- 各级别条数 ---");
for (level, n) in &counts {
    writeln!(w, "{level}: {n}");
}
writeln!(w, "--- ERROR 详情(共 {} 条)---", errors.len());
for e in &errors {
    writeln!(w, "[{}] {}", e.timestamp, e.message);
}
w.flush()?; // 显式刷盘

// ---- 加分:JSON 版本汇总(覆盖 serde_json) ----
#[derive(serde::Serialize)]
struct Summary {
    total: u32,
    bad_lines: u32,
    counts: HashMap<String, u32>,
    error_count: usize,
}

let summary = Summary {
    total,
    bad_lines,
    counts,
    error_count: errors.len(),
};

let json_path = format!("{}", out_path);
fs::write(&json_path, serde_json::to_string_pretty(&summary)?);

println!("完成:汇总写入 {out_path},JSON 写入 {json_path}");

```

```
    Ok(())  
}
```

设计说明: `parse_line` 用 `splitn(2, ' ')` 只拆一次, 保证消息里的空格不被切断——这是日志解析的生产常识; 坏行不 `panic` 而是计数, 保证工具在脏数据面前不崩溃; 所有 IO 通过 `?` 传播, `main` 返回 `Result`, 失败时系统打印完整错误并退出非零。试着删掉 `is_file` 检查, 对比“打开失败”的错误信息形态, 理解两种错误处理风格的差异。

下一章预告: 第8章 网络通信——C 的 `socket` 三件套(`socket/bind/connect`)与 Rust 的 `std::net`; TCP/UDP、HTTP 客户端、粘包拆包与最小服务端, 一次讲清。